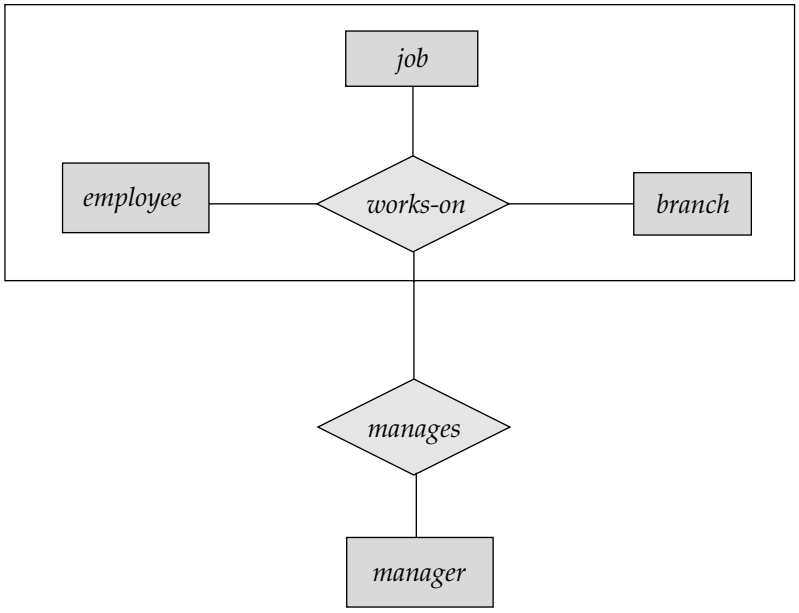


Figure 2.19 E-R diagram with aggregation.

Cardinality constraints can be indicated in several different ways, as Figure 2.21 shows. The labels $*$ and 1 on the edges out of the relationship are sometimes used for depicting many-to-many, one-to-one, and many-to-one relationships, as the figure shows. The case of one-to-many is symmetric to many-to-one, and is not shown. In another alternative notation in the figure, relationship sets are represented by lines between entity sets, without diamonds; only binary relationships can be modeled thus. Cardinality constraints in such a notation are shown by "crow's foot" notation, as in the figure.

2.8 Design of an E-R Database Schema

The E-R data model gives us much flexibility in designing a database schema to model a given enterprise. In this section, we consider how a database designer may select from the wide range of alternatives. Among the designer's decisions are:

- Whether to use an attribute or an entity set to represent an object (discussed earlier in Section 2.2.1)
- Whether a real-world concept is expressed more accurately by an entity set or by a relationship set (Section 2.2.2)
- Whether to use a ternary relationship or a pair of binary relationships (Section 2.2.3)

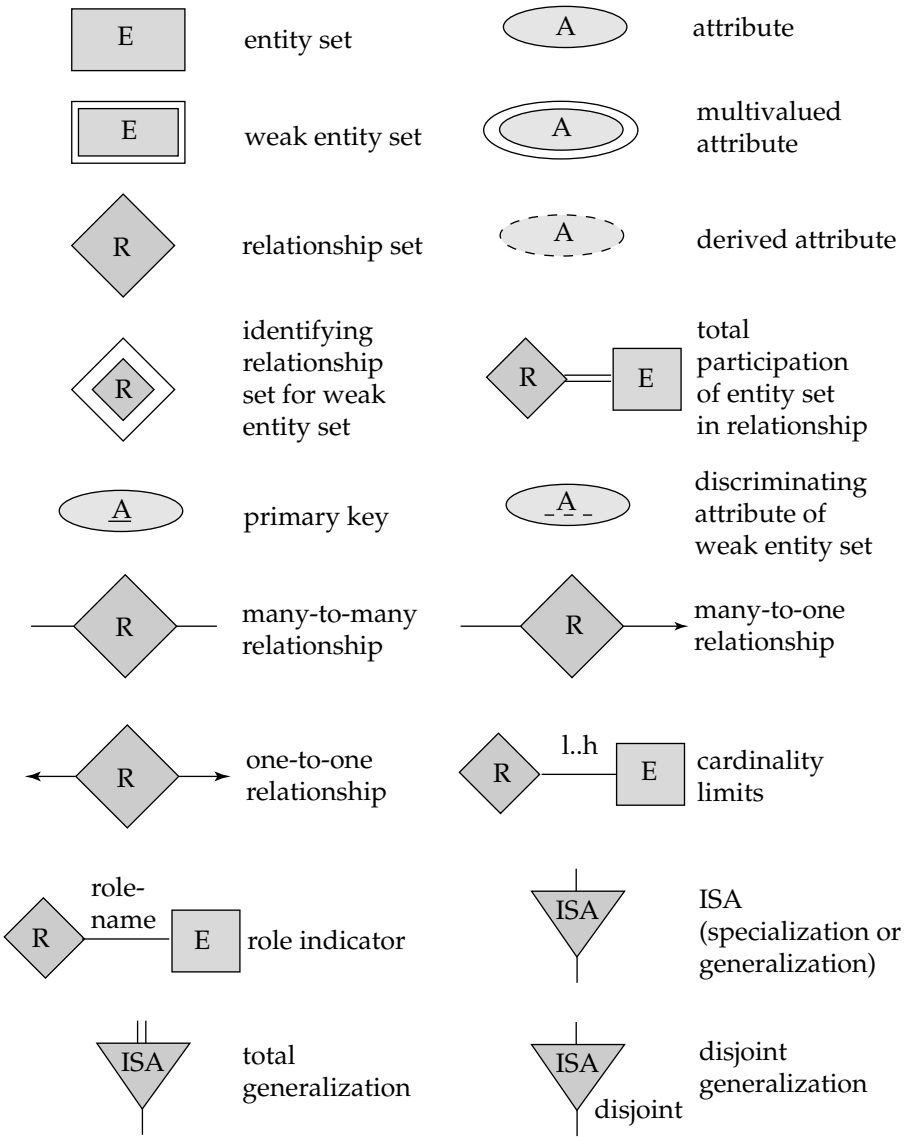


Figure 2.20 Symbols used in the E-R notation.

- Whether to use a strong or a weak entity set (Section 2.6); a strong entity set and its dependent weak entity sets may be regarded as a single "object" in the database, since weak entities are existence dependent on a strong entity
- Whether using generalization (Section 2.7.2) is appropriate; generalization, or a hierarchy of ISA relationships, contributes to modularity by allowing com-

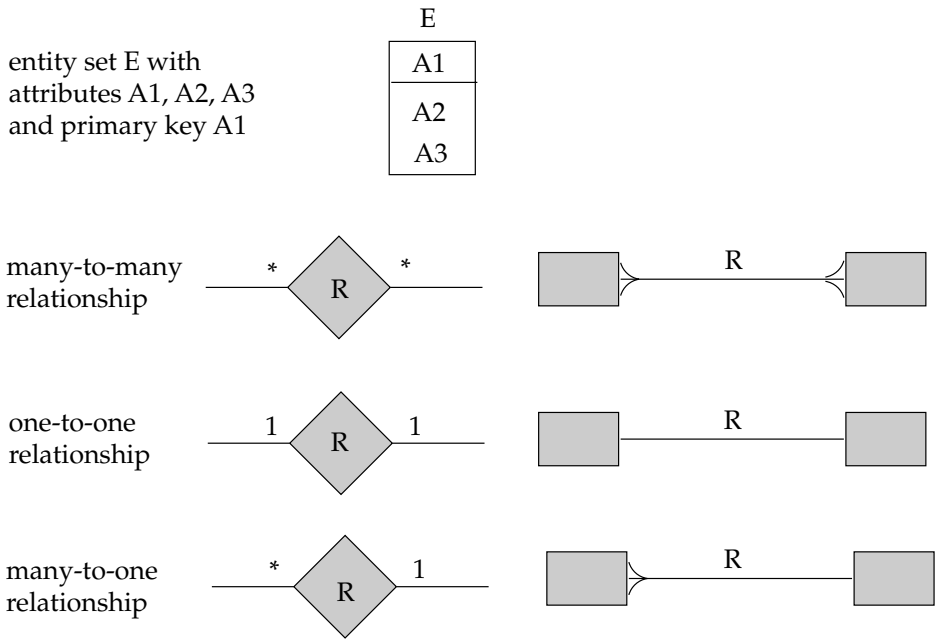


Figure 2.21 Alternative E-R notations.

mon attributes of similar entity sets to be represented in one place in an E-R diagram

- Whether using aggregation (Section 2.7.5) is appropriate; aggregation groups a part of an E-R diagram into a single entity set, allowing us to treat the aggregate entity set as a single unit without concern for the details of its internal structure.

We shall see that the database designer needs a good understanding of the enterprise being modeled to make these decisions.

2.8.1 Design Phases

A high-level data model serves the database designer by providing a conceptual framework in which to specify, in a systematic fashion, what the data requirements of the database users are, and how the database will be structured to fulfill these requirements. The initial phase of database design, then, is to characterize fully the data needs of the prospective database users. The database designer needs to interact extensively with domain experts and users to carry out this task. The outcome of this phase is a specification of user requirements.

Next, the designer chooses a data model, and by applying the concepts of the chosen data model, translates these requirements into a conceptual schema of the database. The schema developed at this **conceptual-design** phase provides a detailed overview of the enterprise. Since we have studied only the E-R model so far, we shall

use it to develop the conceptual schema. Stated in terms of the E-R model, the schema specifies all entity sets, relationship sets, attributes, and mapping constraints. The designer reviews the schema to confirm that all data requirements are indeed satisfied and are not in conflict with one another. She can also examine the design to remove any redundant features. Her focus at this point is describing the data and their relationships, rather than on specifying physical storage details.

A fully developed conceptual schema will also indicate the functional requirements of the enterprise. In a **specification of functional requirements**, users describe the kinds of operations (or transactions) that will be performed on the data. Example operations include modifying or updating data, searching for and retrieving specific data, and deleting data. At this stage of conceptual design, the designer can review the schema to ensure it meets functional requirements.

The process of moving from an abstract data model to the implementation of the database proceeds in two final design phases. In the **logical-design phase**, the designer maps the high-level conceptual schema onto the implementation data model of the database system that will be used. The designer uses the resulting system-specific database schema in the subsequent **physical-design phase**, in which the physical features of the database are specified. These features include the form of file organization and the internal storage structures; they are discussed in Chapter 11.

In this chapter, we cover only the concepts of the E-R model as used in the conceptual-schema-design phase. We have presented a brief overview of the database-design process to provide a context for the discussion of the E-R data model. Database design receives a full treatment in Chapter 7.

In Section 2.8.2, we apply the two initial database-design phases to our banking-enterprise example. We employ the E-R data model to translate user requirements into a conceptual design schema that is depicted as an E-R diagram.

2.8.2 Database Design for Banking Enterprise

We now look at the database-design requirements of a banking enterprise in more detail, and develop a more realistic, but also more complicated, design than what we have seen in our earlier examples. However, we do not attempt to model every aspect of the database-design for a bank; we consider only a few aspects, in order to illustrate the process of database design.

2.8.2.1 Data Requirements

The initial specification of user requirements may be based on interviews with the database users, and on the designer's own analysis of the enterprise. The description that arises from this design phase serves as the basis for specifying the conceptual structure of the database. Here are the major characteristics of the banking enterprise.

- The bank is organized into branches. Each branch is located in a particular city and is identified by a unique name. The bank monitors the assets of each branch.

- Bank customers are identified by their *customer-id* values. The bank stores each customer's name, and the street and city where the customer lives. Customers may have accounts and can take out loans. A customer may be associated with a particular banker, who may act as a loan officer or personal banker for that customer.
- Bank employees are identified by their *employee-id* values. The bank administration stores the name and telephone number of each employee, the names of the employee's dependents, and the *employee-id* number of the employee's manager. The bank also keeps track of the employee's start date and, thus, length of employment.
- The bank offers two types of accounts—savings and checking accounts. Accounts can be held by more than one customer, and a customer can have more than one account. Each account is assigned a unique account number. The bank maintains a record of each account's balance, and the most recent date on which the account was accessed by each customer holding the account. In addition, each savings account has an interest rate, and overdrafts are recorded for each checking account.
- A loan originates at a particular branch and can be held by one or more customers. A loan is identified by a unique loan number. For each loan, the bank keeps track of the loan amount and the loan payments. Although a loan-payment number does not uniquely identify a particular payment among those for all the bank's loans, a payment number does identify a particular payment for a specific loan. The date and amount are recorded for each payment.

In a real banking enterprise, the bank would keep track of deposits and withdrawals from savings and checking accounts, just as it keeps track of payments to loan accounts. Since the modeling requirements for that tracking are similar, and we would like to keep our example application small, we do not keep track of such deposits and withdrawals in our model.

2.8.2.2 Entity Sets Designation

Our specification of data requirements serves as the starting point for constructing a conceptual schema for the database. From the characteristics listed in Section 2.8.2.1, we begin to identify entity sets and their attributes:

- The *branch* entity set, with attributes *branch-name*, *branch-city*, and *assets*.
- The *customer* entity set, with attributes *customer-id*, *customer-name*, *customer-street*; and *customer-city*. A possible additional attribute is *banker-name*.
- The *employee* entity set, with attributes *employee-id*, *employee-name*, *telephone-number*, *salary*, and *manager*. Additional descriptive features are the multivalued attribute *dependent-name*, the base attribute *start-date*, and the derived attribute *employment-length*.

- Two account entity sets—*savings-account* and *checking-account*—with the common attributes of *account-number* and *balance*; in addition, *savings-account* has the attribute *interest-rate* and *checking-account* has the attribute *overdraft-amount*.
- The *loan* entity set, with the attributes *loan-number*, *amount*, and *originating-branch*.
- The weak entity set *loan-payment*, with attributes *payment-number*, *payment-date*, and *payment-amount*.

2.8.2.3 Relationship Sets Designation

We now return to the rudimentary design scheme of Section 2.8.2.2 and specify the following relationship sets and mapping cardinalities. In the process, we also refine some of the decisions we made earlier regarding attributes of entity sets.

- *borrower*, a many-to-many relationship set between *customer* and *loan*.
- *loan-branch*, a many-to-one relationship set that indicates in which branch a loan originated. Note that this relationship set replaces the attribute *originating-branch* of the entity set *loan*.
- *loan-payment*, a one-to-many relationship from *loan* to *payment*, which documents that a payment is made on a loan.
- *depositor*, with relationship attribute *access-date*, a many-to-many relationship set between *customer* and *account*, indicating that a customer owns an account.
- *cust-banker*, with relationship attribute *type*, a many-to-one relationship set expressing that a customer can be advised by a bank employee, and that a bank employee can advise one or more customers. Note that this relationship set has replaced the attribute *banker-name* of the entity set *customer*.
- *works-for*, a relationship set between *employee* entities with role indicators *manager* and *worker*; the mapping cardinalities express that an employee works for only one manager and that a manager supervises one or more employees. Note that this relationship set has replaced the *manager* attribute of *employee*.

2.8.2.4 E-R Diagram

Drawing on the discussions in Section 2.8.2.3, we now present the completed E-R diagram for our example banking enterprise. Figure 2.22 depicts the full representation of a conceptual model of a bank, expressed in terms of E-R concepts. The diagram includes the entity sets, attributes, relationship sets, and mapping cardinalities arrived at through the design processes of Sections 2.8.2.1 and 2.8.2.2, and refined in Section 2.8.2.3.